

AI Interaction Progress

Checkpoint 2



Table of Contents

Table of Contents **2**
AI Interaction Summary 3
Motor Control Key Prompts + Problems 5
Bluetooth Communication Key Prompts + Problems 6
IR RPM Measurement Key Prompts + Problems 9
AI Software Documentation Development 9
AI Hardware Documentation Development 10

AI Interaction Summary

AI Interaction	**AI Contribution (all ChatGPT except where stated)**	**Confirmation**
Motor Control (ARM assembly and C code)	Generated motorDriver.s and main_chkp1.c	Successfully controlled motor using keyboard commands and observed expected motor behavior as per the checksheet
iOS development environment on the MacBook	Step-by-step instructions on Xcode project setup and setting the iPhone in developer mode	Successfully ran and deployed PARMCO app on iPhone
Swift UI Interface	Created buttons, display UI, and required displays and ack logs	Interacted with the buttons and double checked labels and all required buttons were present, PARMCO app was responsive to pressing buttons
CoreBluetooth	Generated BluetoohManager.swift	Xcode build was successful, app launched. Confirmed after RP4 bluetooth configuration
RP4 BLE Configuration	Generated the GLib/GIO dependencies and C implementation	Installation was completed successfully, and the C program compiled as well. When tested with the PARMCO app, successfully connected RP4 to the app
Two way communication with phone and RP4	Generated c file, parmco_ble.c	Terminal output showed the ACK message corresponding to the button pressed
Issue with phone acknowledgement	Suggested reconnecting the bluetooth, potential glitch	Post reconnection, iPhone received corresponding ACK message from RP4
IR sensor code	Generated new main_cp2_ir.c including calculation of IR RPM	Completed a code review to ensure the IR RPM calculation was implemented correctly. Completed test A and observed IR RPM matching target RPM & manual RPM calculation.
Hardware Documentation	Generating and refining the hardware schematics, parts list, and “how it works” explanation documents. After grading feedback on checkpoint 1, prompts were	Successfully implemented the hardware and validated the connections suggested by the schematic with a multimeter and compared with the datasheets for the designated components.

	provided to refine the documents to get closer to a professional level. Additional prompts were used to fit the existing documents to reflect the changes made during checkpoint 2. **Documentation began development in Gemini Pro for Checkpoint 1, and then migrated to Copilot Pro with Claude integration for refinement in later prompts.**	

Motor Control Key Prompts + Problems

Prompt: [ChatGPT Plus] Create a Raspberry Pi 4 motor control program in C and ARM assembly that uses the provided course E4235 library and meets the provided Checkpoint 1 keyboard control requirements. Use the GPIO/wiring structure in the attached circuit schematic pdf. The required functionality includes enabling/disabling the H-bridge, selecting direction, commanding one revolution, continuous rotation, adjusting step rate, and displaying motor status, all controlled by keyboard inputs.
Result: Generated motorDriver_AArch64_version.S and PARMCO_CP1_Original_main.c initial code versions, with keyboard commands, H-Bridge control, one revolution, continuous rotation, and motor status display present.
Problems: Provided the code in AArch64 assembly instead of ARM32 syntax, C file did not allow speed adjustments for single revolution and started at 20 RPM

Prompt: [ChatGPT Plus] First, correct motorDriver.s so it uses ARM32 syntax correctly. Want to use half step mode instead to observe if the 360 rotation is happening more completely. Add adjustable speed for the one revolution speed so we can speed it up to allow for easier testing. Start the initial speed at 10 RPM to match project requirements.
Result: Generated fully corrected motorDriver.s in ARM32 assembly and main_chkp1.c c file with corrections. Both files now meet the checkpoint 1 requirements and function as expected when tested with the hardware. The hardware had issues initially because the propeller blade was not fully pushed down on the motor, after fixing that, code worked perfectly in half-step mode.
Problems: None related to the code, worked as expected.

Bluetooth Communication Key Prompts + Problems

Prompt: [ChatGPT Plus] Now I want to move on to checkpoint 2, specifically working on the Mac to set up the environment for bluetooth connection between the iPhone and RP4. What are the next steps I should take to do this setup on the Mac? I have provided the specific checkpoint 2 requirements, and please ask me for the class library document if you plan to generate code.
Result: Step-by-step instructions detailing how to set up Xcode settings, and initial settings and App.swift
Problems: Steps did not match new Xcode interface, provided screenshots of the UI and provided correct working steps to set up with the new interface

Prompt: [ChatGPT Plus] Provided screenshots of issues at each step, including project setup, setting the iPhone in developer mode, etc.
Result: Provided detailed Xcode project setup, certificate access, created Xcode project with ContentView.swift and PARMCOApp.swift, and initial “Hello World” code deployment onto the iPhone
Problems: Issues encountered with setup, resolved when screenshots were provided

Summary of setup issues		
Issue	Details	AI Result
Invalid bundle identifier	Original project produced Failed Registering Bundle Identifier and No profiles ... were found	New standard iOS project named PARMCO created instead of swift app
Code signing permission	The Mac displayed codesign wants to access the key "Apple Development..."	Mac keychain password is just the login password, enable allow option
Developer mode disabled	Xcode displayed Developer Mode disabled	Steps to enable Developer Mode on the iPhone in Settings → Privacy & Security

Prompt: [ChatGPT Plus] Let's improve ContentView.swift to contain speed, direction, start/stop, and Bluetooth status indicators on the app UI. Motor control is not required for checkpoint 2. Provide instructions on deploying this version on the iPhone to verify functionality.
Result: Generated ContentView.swift , and steps for phone deployment:

Step 2: Deploy the updated interface

1. Connect the iPhone to the Mac with USB-C.
2. Unlock the iPhone.
3. Select the physical iPhone as the Xcode run destination.
4. Press ▶️ **Run** or `Command + R`.
5. Verify that the updated PARMCO interface appears on the phone.
6. Test Faster, Slower, Direction, and Start/Stop to confirm that the displayed values respond.

Problems: No issues, just move on to the next step.

Prompt: [ChatGPT Plus] Deployed on phone successfully, give me instructions on how to add bluetooth communication, following project guidelines. Focus on using Swift's core Bluetooth framework to support Bluetooth connection to the RP4. Include scanning for the PARMCO BLE service, connecting, receiving notifications, and disconnecting.

Result: Generated BluetoothManager.swift , specified adding Xcode's Target → Info settings to add Privacy - Bluetooth Always Usage Description to project settings. Specified next steps as compiling the file and then deploying the app to the iPhone again.

Problems: Issues/confusion with adding the Target → Info setting, asked for further clarification from ChatGPT which resolved the issue

Prompt: [ChatGPT Plus] Now we need to set up the Raspberry Pi 4 to be able to connect to the app, using the RP4 built in bluetooth. Write the code in C for the RP4 to be able to communicate with the iPhone and support two way communication. Make sure RP4 prints received messages and sends acknowledgement to the iPhone app.

Result: Before generating, requested to run the following commands and provide output.
cat /etc/os-release | grep PRETTY_NAME
bluetoothctl --version
systemctl is-active bluetooth
bluetoothctl show
pkg-config --modversion gio-2.0

Problems: None

Prompt: [ChatGPT Plus] Here is the output of running the suggested commands, suggest next steps and create the code file:
group1@raspberrypi:~ $ cat /etc/os-release | grep PRETTY_NAME
PRETTY_NAME="Raspbian GNU/Linux 12 (bookworm)"
group1@raspberrypi:~ $ systemctl is-active bluetooth
active
group1@raspberrypi:~ $ bluetoothctl --version
bluetoothctl: 5.66

group1@raspberrypi:~ $ pkg-config --modversion gio-2.0 Package gio-2.0 was not found in the pkg-config search path.
Result: Generated parmco_ble.c with compilation instructions and provided installation commands for GLib/GIO development files needed for the RP4 bluetooth communication: sudo apt update sudo apt install -y libglib2.0-dev pkg-config,
Problems: None, the C file ran as expected working with the mobile app

IR RPM Measurement Key Prompts + Problems

Prompt: [ChatGPT Plus] This is the IR sensor, here is the previous code and class library that you need to make updates to the code. Make sure the code includes functionality for the IR sensor RPM as specified: The C program will read the sensor's digital output using the course GPIO library, detect pulse edges, timestamp them, calculate the actual RPM, and reset the measurement to zero after the motor stops or no pulses arrive for an appropriate timeout, as specified by the instructions.
Result: Generation of main_cp2_ir.c, which after testing and troubleshooting blade issues (needed to add reflective tape on the blades, suggestion from Zac Brammer), code worked as expected with no changes required. Hardware accuracy was human-verified.
Problems: Initially did not work because of lack of reflective tape and the metal of the IR sensor touching the metal of the stepper motor, but was not a code/AI issue.

AI Software Documentation Development

Prompt: [ChatGPT Plus] I need to create software documentation with library attribution, how each library works and why it is needed. Code flags must also be present. This document should cover documentation for main_cp2_ir.c, parmco_ble.c, motorDriver.s and the PARMCO app files (PARMCOapp.swift, BluetoothManager.swift, ContentView.swift), all of which are attached. Generate a pdf with these details, ensuring documentation is consistent with the provided code behavior. Request additional documents/information required, but do not hallucinate results.
Result: Generated as a 13-page pdf, covering responsibilities, libraries, architecture, function behavior, and compiler attribution. Requested 4 things: confirmation of IR sensor,
Problems: Had unprofessional/unnecessary sections and descriptions, used incorrect file names, requested additional context for confirmation/completion.

Prompt: [ChatGPT Plus] Remove the "important grading interpretation" on the first page. use the file names main_cp2_ir.c, motorDriver.s, and parmco_ble.c. Just say "class library credit" instead of "Required class library credit" you have the history of the Xcode project settings, attached is the zip file if any additional information is required. Do not have references to the code project guide such as "course project guide (page 4)" and remove "AI-generated, verify before submission" I need a clean PDF software documentation ready for submission. remove the validation and claims ledger, and items needed before final submission sections. I can confirm Test A and motor control demonstration work as expected. The only human edit to the code was changing a GPIO for 4N25 enable wire from GPIO 4 to GPIO 26. Here is the github link.
Result: Generated the corrected PARMCO_CP2_Software_Documentation_Final.pdf as a 9-page pdf, incorporated the Xcode settings, correct filenames, etc.
Problems: None, PDF generated as desired.

AI Hardware Documentation Development

Prompts for Checkpoint 2 Hardware Documentation:

Prompt: [Copilot Pro With Claude Integration] Update the HBridge Circuit Analysis to be correct through checkpoint 2 functionality. The final document will be a "correct how it works analysis" for the circuit. Add onto the already existing information in the PARMCO_Hbridge attached document and just add the new Checkpoint 2 functionality. (With updated main method for Checkpoint 2 functionality code attached in the prompt)
Result: Generated the PARMCO_Circuit_Analysis_"How_it_Works"_CP2.pdf as a pdf, adding on the new explanation of the additional IR wiring components and the new UI features. It incorporated the specific checkpoint 2 functionality and updated the file to reflect the newest code functionality.
Problems: None, PDF generated just fine.

Prompt: [Copilot Pro With Claude Integration] Make this schematic more detailed and professional. keep all GPIO assignments and wiring connections the same just make the connections more detailed and professional**. (Attached the hardware schematic submitted for checkpoint 1)**
Result: Generated the PARMCO_Schematic_Detailed.pdf and reflected the request for a cleaner, more professional look while keeping all original connections the same. The critique we received for the grading of the first checkpoint hardware documentation was that it should be a bit more detailed and professional so that wording was used in the prompting and the result was improved as a result. It was validated against the previous version to ensure no hallucination or GPIO removal happened in the new generation.
Problems: None, PDF generated just fine.

Prompts for Checkpoint 1 Hardware Documentation:

Prompt: [Gemini Pro] Make a new schematic with detailed drawings and pinout assignments for these documents (Raspberry_Pi_4_Stepper_Motor_Schematic.pdf)
Result: Generated the PARMCO_Schematic_Detailed.pdf and reflected the request for a cleaner, more professional look while keeping all original connections the same. The critique we received for the grading of the first checkpoint hardware documentation was that it should be a bit more detailed and professional so that wording was used in the prompting and the result was improved as a result. It was validated against the previous version to ensure no hallucination or GPIO removal happened in the new generation.
Problems: None, PDF generated just fine.

Prompt: [Gemini Pro] Based on these documents build me a simple schematic for how to wire connect the H bridge, octopolar, RP4, stepper motor and anything else needed. Label the

schematic with pin numbers and as much detail as possible (attached is the PARMCO Project directions PDF and the datasheet for the HBridge)
Result: Generated the Raspberry_Pi_4_Stepper_Motor_Schematic.pdf which served as the first iteration of the schematic we would tweak through the prompts above. It did well at establishing the connections on the board that we continued to use through checkpoint 2 and had no errors while testing. The only edit needed to be made was the change of GPIO 04 and that was a design choice due to the fact that wire was becoming cumbersome in out wiring setup and we wanted to move it just to create more ease of access.
Problems: We switched the GPIO 04 connection to GPIO 26 after this prompt so the prompt was regenerated with the same exact file but told to just switch that single connection.

Prompt: [Copilot Pro with Claude Integration] make an easy to understand hardware parts list as well (Attached is the Raspberry_Pi_4_Stepper_Motor_Schematic.pdf)
Result: The parts list PARMCO_parts_list.pdf that is used for the checkpoint 1 and checkpoint 2 parts list requirement. For checkpoint 1 we wired all components, including the IR sensor we didn't implement until checkpoint 2 so this parts list continues to be accurate as we have not introduced any new hardware into the circuit or schematic.
Problems: None, PDF generated just fine.